REPOSITORY SUMMARY

Customer Support AI Chatbot — Repository Assessment

Abhinav-kanduri/Customer-Support-AI-Chatbot | feature/july26th

Repository	Abhinav-kanduri/Customer-Support-AI-Chatbot
Repository URL	https://github.com/Abhinav-kanduri/Customer-Support-AI-Chatbot
Description	Build an AI chatbot for a SaaS or e-commerce company that can understand customer messages, classify intent, retrieve company-specific knowledge, answer with citations, call business tools, create tickets, and escalate risky or unresolved cases to humans.
Default branch	main
Analyzed ref	feature/july26th
Commit SHA	586605e0d70bbb7413a043d5fb929b05674282c5
Visibility	public
Generated	2026-08-09 18:19 UTC
Model	gpt-5.6-terra
Files analyzed	46 of 71
Characters analyzed	296,450
Summary batches	9

Executive Summary

At commit 586605e0d70bbb7413a043d5fb929b05674282c5 on feature/july26th, the repository contains a partially implemented FastAPI support-service baseline, two separate Streamlit applications, and a comprehensive but explicitly planned PI-1 architecture/migration package. Implemented backend capabilities include direct OpenAI chat, basic credential lookup, PostgreSQL connectivity diagnostics, read-only escalation-record retrieval, and document ingestion with OpenAI embeddings stored in PostgreSQL/pgvector. However, the implemented chat path is stateless and not connected to retrieval, citations, intent classification, guardrails, conversation persistence, or escalation creation. The intended end-to-end support-agent workflow is well specified in PI-1 documentation and draft SQL but must not be treated as deployed behavior.

Problem Statement

The repository aims to provide a SaaS/e-commerce customer-support chatbot that understands messages, classifies intent, retrieves organization-specific knowledge, answers with citations, routes risky or unresolved cases to people, and captures operational feedback. The current implementation establishes selected building blocks but does not yet deliver this workflow as one secured, grounded, auditable service.

Primary Capabilities

- FastAPI service bootstrap with root, health, database diagnostic, chat, authentication, document, and escalation routers.
- Stateless direct support chat using OpenAI Chat Completions with gpt-4o.
- PDF, DOCX, and TXT ingestion; extracted-text persistence; chunking; OpenAI text-embedding-3-small embedding; and pgvector-backed chunk storage.
- Basic username/password lookup against public.app_users that returns a stored role.
- Read-only listing of rows from public.escalated_table.
- Standalone Streamlit restaurant-support classifier that maps text to one of 24 fixed intent codes, departments, recommended actions, and escalation flags.
- Standalone Streamlit RAG-quality dashboard using deterministic generated/demo data, not live telemetry.
- PI-1 planned design for stateful orchestration, idempotency, NLU, policy routing, RAG retrieval with validated citations, guardrails, escalation-case management, feedback, telemetry, and evaluations.

Architecture Overview

- The implemented backend is a modular FastAPI application assembled in app/main.py, with route modules for authentication, chat, documents, escalations, and database diagnostics.
- The current /chat route directly sends a user prompt and fixed customer-support instruction to OpenAI; it does not invoke the classifier, retrieve document chunks, preserve conversational state, cite sources, create tickets, or escalate cases.
- The document pipeline persists extracted source text in PostgreSQL, creates 1,536-dimensional embeddings, and stores chunks in pgvector with cosine HNSW indexing. No implemented similarity-search endpoint or chat-to-retrieval connection is evidenced.
- The restaurant intent classifier is an independent Streamlit application using local scikit-learn/joblib artifacts; it is not integrated with FastAPI.

- The RAG Quality Dashboard in root app.py is another independent Streamlit surface and explicitly uses fabricated/generated data only.
- PI-1 defines a target modular-monolith pipeline: authenticated/idempotent request handling; conversation/message persistence; input guardrails and PII masking; NLU; deterministic policy routing; retrieval and grounded generation where applicable; output validation; human escalation; and trace-correlated telemetry and feedback.
- PI-1 SQL migrations define a proposed PostgreSQL persistence model and depend on numeric migration order. Documentation explicitly identifies them as draft and unapplied.

Technology Stack

- Python; FastAPI 0.115.5; Uvicorn.
- OpenAI Python SDK 1.82.1, currently using gpt-4o for chat and text-embedding-3-small for embeddings.
- PostgreSQL/Supabase-oriented persistence through psycopg 3.2.13.
- pgvector vector(1536) storage and cosine-distance HNSW indexes.
- pypdf and python-docx for document text extraction; python-multipart for uploads.
- Pydantic request/response models.
- Streamlit, pandas, NumPy, Plotly, scikit-learn, and joblib for the standalone classifier and dashboards.
- Railway/Nixpacks deployment configuration and GitHub Actions workflow.
- python-dotenv configuration loading; optional/reserved Supabase URL, publishable-key, and JWKS settings.

Repository Languages

- Python: 21,677 bytes

Key Components

FastAPI application bootstrap and health

Initializes configuration before application imports, mounts route modules, records process uptime, exposes liveness-oriented root/health responses, and provides a database diagnostic endpoint.

Paths: app/main.py, app/config.py, app/database.py

Direct OpenAI chat

Accepts a single prompt and returns a gpt-4o completion. It is a direct, stateless LLM call rather than a connected RAG or support-orchestration workflow.

Paths: app/routes/chat.py

Authentication lookup

Checks supplied username/password values against public.app_users and returns a YES/NO result and stored role. It does not establish a token, session, or endpoint authorization context.

Paths: app/routes/auth.py

Document ingestion and embedding

Validates PDF, DOCX, and TXT uploads; extracts text; persists document records and states; chunks content; creates OpenAI embeddings; and stores vectors in PostgreSQL/pgvector.

Paths: app/routes/documents.py, sql/document_chunks.sql

Escalated-record reader

Returns rows from the legacy-style public.escalated_table. It is read-only and does not implement case creation, assignment, status transitions, or conversation linkage.

Paths: app/routes/escalated.py

Restaurant intent-classifier application

Loads a serialized scikit-learn model to classify restaurant-support messages into 24 constrained intent codes and produce routing/escalation guidance for single messages and CSV batches.

Paths: restaurant_intent_streamlit_app/mlapp.py, restaurant_intent_streamlit_app/artifacts/, datasets/ml_model/restaurant_intent_multi_algorithm_training (1).py

RAG Quality Dashboard

Renders illustrative RAG pipeline, retrieval, answer-quality, and roadmap views from generated in-memory data. It does not monitor the backend, database, vector store, LLM, or live quality metrics.

Paths: app.py

PI-1 target platform specification

Specifies planned stateful chat orchestration, reusable NLU, deterministic routing, grounded RAG, guardrails, escalation workflows, feedback, evaluations, and observability.

Paths: PI_1/README.md, PI_1/Feature_1.md, PI_1/Feature_2.md, PI_1/Feature_3.md, PI_1/Feature_4.md, PI_1/Feature_5.md

PI-1 draft schema/migrations

Defines proposed relational entities, constraints, retrieval functions, policy versioning, escalation lifecycle, and telemetry/evaluation storage. It is documented as draft/non-deployed.

Paths: PI_1/sql/001_conversation_orchestrator.sql, PI_1/sql/002_nlu_routing.sql, PI_1/sql/003_rag_retrieval.sql, PI_1/sql/004_guardrails_escalation.sql, PI_1/sql/005_feedback_observability.sql, PI_1/DATABASE_SCHEMA.md

Delivery automation and governance

Documents local setup, Railway environments, branch promotion policy, linting, and deployment configuration. Some deployment documentation conflicts with the actual workflow characterization and requires reconciliation.

Paths: .github/workflows/deployment.yml, railway.toml, SETUP.md, RAILWAY_DEPLOYMENT.md, DEPLOYMENT_RULES.md, CONTRIBUTING.md

API Endpoints

GET /

Implemented service-running confirmation.

Source: app/main.py

GET /health

Implemented liveness-oriented health response with uptime and database availability; status remains "ok" even when the dependency is unavailable.

Source: app/main.py

GET /database/test

Implemented PostgreSQL diagnostic returning database identity, database user, and server version; connection failures return 503.

Source: app/database.py

POST /auth/login

Implemented credential lookup against public.app_users and return of stored role; does not issue authentication tokens.

Source: app/routes/auth.py

POST /chat

Implemented stateless direct OpenAI gpt-4o support prompt completion.

Source: app/routes/chat.py

POST /documents/upload

Implemented PDF/DOCX/TXT upload, text extraction, and creation of a RECEIVED document record.

Source: app/routes/documents.py

POST /documents/chunks

Implemented embedding and upsert of one supplied text chunk using optional/generated document and chunk IDs.

Source: app/routes/documents.py

POST /documents/{doc_id}/process

Implemented synchronous document chunking, embedding, vector persistence, and lifecycle-state updates.

Source: app/routes/documents.py

GET /escalated

Implemented unpaginated read of all selected public.escalated_table rows.

Source: app/routes/escalated.py

POST /v1/chat

Planned PI-1 stateful, idempotent chat endpoint returning conversation/message IDs, response, intent, citations, next action, escalation state, and trace ID.

Source: PI_1/Feature_1.md

POST /v1/search

Planned internal retrieval endpoint returning ranked document chunks.

Source: PI_1/Feature_3.md

GET /v1/escalations

Planned role-protected escalation queue.

Source: PI_1/Feature_4.md

PATCH /v1/escalations/{case_id}

Planned authorized escalation assignment/status transition endpoint with immutable event history.

Source: PI_1/Feature_4.md

POST /v1/feedback

Planned authorized assistant-message feedback endpoint retaining audit history.

Source: PI_1/Feature_5.md

Data and Storage

- Implemented runtime tables include public.app_users for credential/role lookup, public.documents for extracted text and processing status, public.document_chunks for chunk text and vectors, and public.escalated_table for legacy escalation/order fields. Migrations for app_users, documents, and escalated_table are not evidenced.
- public.document_chunks is defined with a unique (doc_id, chunk_id) constraint, vector(1536), and a cosine HNSW index. The standalone DDL does not demonstrate tenant/ownership fields, RLS, retention procedures, or automatic update-time maintenance.
- Current document processing retains extracted raw text in PostgreSQL but does not retain original uploaded files in object storage, limiting re-extraction and source-fidelity options.
- The direct chat flow does not persist conversations, messages, retrieved sources, classifier results, or model-call audit records.
- PI-1 proposes support_users, conversations, messages, pipeline_runs, idempotency_records, NLU/routing records, documents/chunks, retrieval events/results, citations, guardrail and PII records, escalation cases/events, feedback, application events, and evaluation entities.
- PI-1 data handling is designed around redacted content by default, optional approved encrypted originals, hashed identifiers, safe JSONB metadata, and no raw PII values in PII-detection records. Final retention policy remains pending approval.
- Restaurant classifier artifacts are duplicated across dataset and Streamlit artifact locations. Metadata includes a Windows-style artifact path, creating portability and version-selection ambiguity.

Request and Processing Flow

- Current chat: POST /chat validates the prompt, sends a fixed system instruction and caller text to OpenAI gpt-4o, and returns the first generated response. No retrieval, intent routing, citation validation, policy guardrail, or escalation action occurs.
- Current ingestion: upload validates file extension and extracted content, reads the full file into memory, stores extracted text as RECEIVED, and later processes the document synchronously through PROCESSING, CHUNKED, EMBEDDED, or FAILED states.
- Current document processing splits text approximately by paragraphs with a maximum chunk size of 1,500 characters, embeds each chunk individually, and writes chunks in

separate operations. Reprocessing produces new random chunk IDs and can accumulate stale/duplicate content.

- Current classifier: Streamlit normalizes input, loads a local model artifact, predicts one constrained restaurant intent, derives department/action/escalation guidance, and optionally processes CSV batches. This remains isolated from the API.
- Current dashboard: Streamlit creates reproducible generated metrics and visualizations; it makes no calls to operational APIs, databases, vector stores, or LLMs.
- Planned PI-1 flow: validate/authenticate/idempotently accept a message; load or create conversation; persist customer message and pipeline run; mask PII and screen input; run NLU; apply deterministic routing; retrieve eligible pgvector chunks for RAG; generate structured output; validate grounding/citations/policy; respond or create a human case; then persist assistant output, audit data, telemetry, and feedback linkage.

Setup and Run

- Backend local setup is documented as Python 3.11+, virtual environment creation, root requirements installation, and uvicorn app.main:app --reload. Root requirements pin FastAPI, Uvicorn, OpenAI, psycopg, multipart support, pypdf, python-docx, and dotenv.
- app.config loads .env, configures logging, and fails application import if OPENAI_API_KEY or DATABASE_URL is missing. Supabase settings are optional/reserved in the reviewed configuration.
- Railway is configured through railway.toml to use Nixpacks and run uvicorn app.main:app --host 0.0.0.0 --port $PORT, with restart-on-failure bounded to three retries.
- The standalone classifier has its own requirements file and is launched with Streamlit. The root RAG dashboard also requires Streamlit/pandas/NumPy/Plotly, but those imports are not included in root requirements; install intent is ambiguous.
- PI-1 draft migrations must be applied in numeric order from 001 through 005 and require pgcrypto; retrieval migration additionally enables/requires pgvector. They are not evidence of runtime availability.
- GitHub Actions enforces reviewed branch-direction rules, installs root dependencies, runs Ruff, and deploys develop/main using Railway environment configuration. The workflow does not run an automated test command despite its lint-and-test job naming.
- Documentation is inconsistent on deployment maturity: some files call deployment jobs placeholders while the reviewed workflow contains Railway deployment steps. Documentation should be aligned with the actual workflow and environment configuration.

Strengths

- The repository clearly separates implemented backend features, standalone applications, dashboard demonstrations, and PI-1 planning material in much of its architecture documentation.
- The current API is modularized by route responsibility, and startup configuration is intentionally initialized before dependent imports.
- Document ingestion includes format validation, text extraction, lifecycle states, failure marking, embedding creation, a vector-dimension check on the single-chunk route, and a pgvector HNSW cosine index.
- SQL values in reviewed database routes use parameter binding.
- PI-1 provides a strong target data and control-plane design: trace IDs, pipeline runs, idempotency, versioned policies, auditable routing, retrieval-result persistence, validated citations, transactional escalation creation, feedback, and evaluation datasets.
- PI-1 safety design is layered: input validation, PII masking, injection/high-risk detection, deterministic routes, grounding/citation checks, output policy checks, safe fallback behavior, and human handoff.
- The classifier constrains model labels to a 24-intent allowlist and maps unknown labels to OUT_OF_SCOPE instead of exposing arbitrary model output.
- The dashboard explicitly labels its data as generated/demo data, reducing the risk that its charts are presented as actual operational measurements.
- The repository has a defined feature-to-develop-to-main promotion model and separate development/production deployment intent.

Risks and Gaps

- Critical: Current authentication is not production-ready. Login appears to compare submitted credentials with stored values, issues no JWT/session, and no endpoint-level authorization or RBAC is evidenced for chat, documents, database diagnostics, or escalation data.
- Critical: The implemented /chat route is an ungrounded direct LLM call. It does not use stored vectors, retrieval, citations, conversation history, intent classification, guardrails, rate limiting, or escalation; it can produce unsupported answers and uncontrolled provider use.
- Critical: Sensitive or operational endpoints appear unauthenticated. /database/test discloses database/user/version data, and /escalated returns all matching records without authorization, pagination, filtering, or ordering.
- High: Provider/parser errors may be returned as raw exception text by chat and document routes, potentially leaking internal, configuration, or third-party implementation details.
- High: Uploads are fully read into memory without evidenced file-size, MIME/content-type, user-quota, or rate-limit controls. Synchronous

one-request-per-chunk embedding risks high latency, worker exhaustion, provider quota pressure, and partial processing.

- High: Document processing has no evidenced transaction spanning status transitions and chunk writes. Mid-process failure can leave partial vectors or inconsistent state. Reprocessing uses random chunk IDs and can retain duplicate/stale embeddings.
- High: The application depends on runtime tables—documents, app_users, escalated_table—whose migrations are not evidenced. This creates schema drift and environment-reproducibility risk.
- High: PI-1 migrations are explicitly draft/unapplied. Their extensive capabilities, including RAG retrieval, guardrails, escalation creation, feedback, and policy enforcement, are not current functionality.
- High: The classifier is restaurant-specific while the core product description targets SaaS/e-commerce support. Its reported 1.0 held-out metrics derive from synthetic/templated data and do not establish real-world quality, calibration, coverage, or production safety.
- Medium: The Streamlit RAG dashboard is not live monitoring despite its production-oriented label. Its generated scores, failures, costs, and quality metrics cannot support claims about production retrieval or answer quality.
- Medium: The classifier loads trusted-required joblib/pickle artifacts without integrity/provenance verification. Duplicate metadata/artifact locations and platform-specific paths complicate release control.
- Medium: The current database access style opens synchronous connections per request, with no connection pool or repository/service boundary evidenced. Route-embedded data access complicates transaction, security, and testing controls.
- Medium: The health endpoint reports status "ok" even when database access is unavailable, and broadly catches dependency errors without shown local logging. Monitoring based only on status can miss readiness failure.
- Medium: Dependency setup is ambiguous for root app.py because its Streamlit/pandas/NumPy/Plotly imports are absent from root requirements.
- Medium: Several design/documentation conflicts require correction: API health response differs from API.md; documented endpoint status does not reconcile cleanly with mounted routers; app.py is described inconsistently; and deployment material differs on whether deployment jobs are placeholders or active.
- Medium: PI-1 migration concerns include prerequisites/order dependencies, a document_chunks doc_id foreign-key gap in reviewed PI-1 RAG DDL, fixed 1,536-dimensional vector assumptions, and migration guidance noting no destructive rollback path.
- Medium: Deployment branch rules support controlled promotion but provide no documented urgent-hotfix path, and branch-protection/Code Owner enforcement is recommended rather than proven active.

Recommended Next Steps

- P0 — Secure externally reachable routes before further feature expansion: replace credential comparison with a vetted password-hash verifier or external OIDC/Supabase JWT integration; issue and validate authenticated principals; enforce RBAC/ownership on documents, conversations, escalations, diagnostics, and feedback; restrict or remove /database/test from public exposure.
- P0 — Make /chat safe immediately: add request body and prompt limits, authentication, per-user/IP rate limits, OpenAI timeouts/retries/cost limits, sanitized client errors, server-side structured logging, and an explicit safe fallback response. Do not market it as RAG until retrieval/citation enforcement exists.
- P0 — Protect document ingestion: enforce maximum file sizes, MIME/content validation, upload quotas, malware/content-scanning policy as appropriate, parser-error sanitization, and authorization. Move embedding processing off the synchronous request path or impose strict bounded processing limits.
- P1 — Establish reproducible schema management: inventory and version migrations for current public.app_users, public.documents, public.escalated_table, and document_chunks; validate existing environments; back up and reconcile schemas; define migration rollback/recovery procedures; and apply PI-1 migrations only after compatibility, security, and data-retention review.
- P1 — Implement a minimal connected RAG slice: add authorized similarity retrieval over active embedded chunks; establish stable document/chunk versioning and replacement semantics; integrate retrieval into chat; return citations tied to retrieved chunks; reject or hand off when evidence is insufficient; and test retrieval/citation behavior with representative documents.
- P1 — Correct persistence reliability: introduce repository/service boundaries and request-scoped pooled database access; make document status/chunk writes transactional where possible; prevent duplicate chunks on reprocessing; define document deletion/version/retention behavior; and add ownership/tenant-aware access controls.
- P1 — Deliver PI-1 in dependency order rather than treating it as complete: first conversation/message/pipeline-run/idempotency and identity migration, then backend NLU/routing integration, then RAG/citations, then guardrails/escalation workflow, then telemetry/feedback/evaluation. Maintain backward-compatible endpoint migration from /chat to versioned APIs.
- P1 — Implement tests and enforce them in CI: unit tests for auth/RBAC, error sanitization, route policy, classifier artifact validation, document parsing/chunking, vector dimension handling, and idempotency; integration tests for database migrations, retrieval/citations, partial-failure recovery, escalation atomicity, and authorization; add the test command to GitHub Actions.
- P2 — Validate or replace the restaurant classifier for the intended business domain: collect real, consented, redacted SaaS/e-commerce support samples; evaluate external

test sets, class imbalance, calibration, low-confidence behavior, mixed intents, adversarial inputs, and drift; version/sign artifacts; and integrate only after policy thresholds and fallback routing are approved.

- P2 — Separate demonstrators from operational products: rename or conspicuously label the RAG dashboard as demo data, isolate its dependencies and deployment, and avoid presenting generated metrics as production evidence. Similarly, make classifier and dashboard launch/dependency instructions unambiguous.
- P2 — Align documentation and operations: reconcile API.md with implementation, clarify app.py ownership, state actual Railway workflow behavior, document runbooks for database/provider outages and hotfixes, verify branch protections/Code Owner configuration, and define readiness versus liveness monitoring semantics.

Evidence Files

- README.md
- ARCHITECTURE.md
- API.md
- SETUP.md
- RAILWAY_DEPLOYMENT.md
- DEPLOYMENT_RULES.md
- CONTRIBUTING.md
- INSTRUCTION.md
- Feature_built/Feature_1_built.md
- Feature_built/Feature_2.md
- Feature_built/Feature_3.md
- Feature_built/Feature_4.md
- Feature_built/Feature_5.md
- app.py
- app/main.py
- app/config.py
- app/database.py
- app/routes/auth.py
- app/routes/chat.py
- app/routes/documents.py
- app/routes/escalated.py

- requirements.txt
- restaurant_intent_streamlit_app/mlapp.py
- restaurant_intent_streamlit_app/README.md
- restaurant_intent_streamlit_app/requirements.txt
- sql/document_chunks.sql
- datasets/ml_model/restaurant_intent_multi_algorithm_training (1).py
- datasets/restaurant_intent_model_artifacts/model_metadata.json
- restaurant_intent_streamlit_app/artifacts/model_metadata.json
- PI_1/README.md
- PI_1/DATABASE_SCHEMA.md
- PI_1/Feature_1.md
- PI_1/Feature_2.md
- PI_1/Feature_3.md
- PI_1/Feature_4.md
- PI_1/Feature_5.md
- PI_1/sql/README.md
- PI_1/sql/001_conversation_orchestrator.sql
- PI_1/sql/002_nlu_routing.sql
- PI_1/sql/003_rag_retrieval.sql
- PI_1/sql/004_guardrails_escalation.sql
- PI_1/sql/005_feedback_observability.sql
- .github/workflows/deployment.yml
- railway.toml
- .claude/settings.json